

***Universitatea Ovidius din Constanta
Facultatea de Matematica si Informatica
Specializarea Informatica***

Ingineria sistemelor software

Curs 9

Conf. univ. dr. Crenguta M. Puchianu
2020



Proiectare software



Modele de proiectare

- ❑ Model de proiectare = descriere a obiectelor si claselor care colaboreaza pentru rezolvarea unei probleme generale de proiectare intr-un context particular.*
- ❑ Avantaje:
 - ❑ reutilizare,
 - ❑ cresterea calitatii produsului software,
 - ❑ imbunatateste documentatia.
- ❑ Structura generala a unui pattern: nume, problema, solutie, consecinte: avantaje si dezavantaje
- ❑ Clasificarea pattern-urilor data de E. Gamma:
 - Modele pentru atribuirea responsabilitatilor astfel incat sa fie indeplinite cele 2 principii foarte importante ale proiectarii: cuplarea slaba si coeziunea inalta a claselor.
 - Modele de proiectare fundamentale
 - Modele de creare
 - Modele de impartire
 - Modele structurale
 - Modele comportamentale
 - Modele de concurenta

*Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison Wesley Professional, 1995.



-

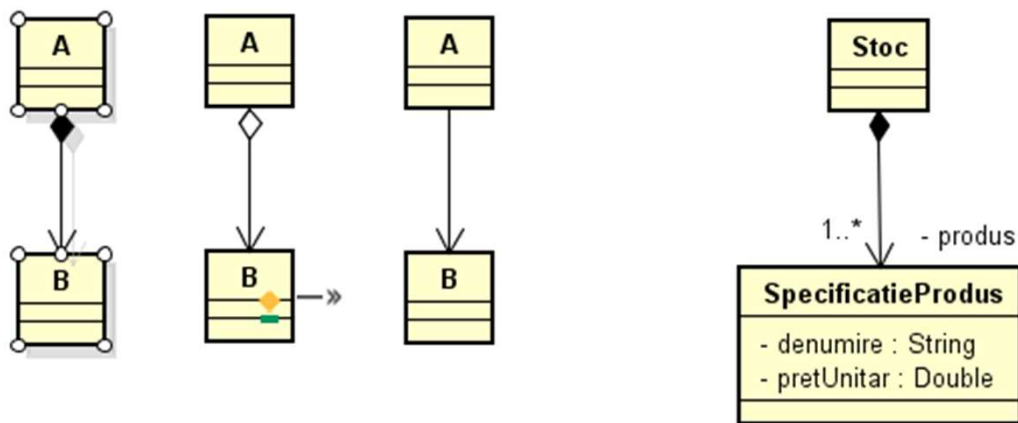
M.P. Creator

Problema: Cine ar trebui sa fie responsabil pentru crearea unui obiect al unei clase?

Solutie: Atribuim clasei A responsabilitatea de a crea un obiect al unui clase B daca una sau mai multe din urmatoarele afirmatii sunt adevarate:

- A *contine* obiecte ale lui B
- A *agrega* obiecte ale lui B
- A *inregistreaza* obiecte ale lui B.
- A *foloseste* obiecte ale lui B.
- A *are date de initializare* ce vor transmise lui B cand este creat (astfel A este un Expert in raport cu crearea lui A).

In aceste cazuri, spunem ca A este un *creator* al obiectelor lui B.



M.P. Information Expert (Expert)

Problema: Care este principiul general de atribuire a responsabilitatilor obiectelor/claselor?

Solutie: Atribuim o responsabilitate acelei clase care are informatiile necesare indeplinirii ei.

Exemplu. Intr-o aplicatie de gestiune a vanzarilor produselor dintr-un magazin, cine este responsabil cu calcularea costului total al unei vanzari?

Dupa pattern-ul Expert, ar trebui sa cautam clasa care are toate informatiile necesare pentru a determina totalul=suma costurilor articolelor cumparate +tva-reducere

Unde cautam clasa? In modelul de domeniu sau in diagrama de clase a arhitecturii software?

Raspunsul este:

Pas 1. Daca arhitectura software curenta din proiectare contine clase, cautam acolo.

Pas 2. Altfel, cautam in diagrama de clase a modelului de domeniu si incercam sa cream clase software inspirandu-ne din clasele modelului de domeniu.

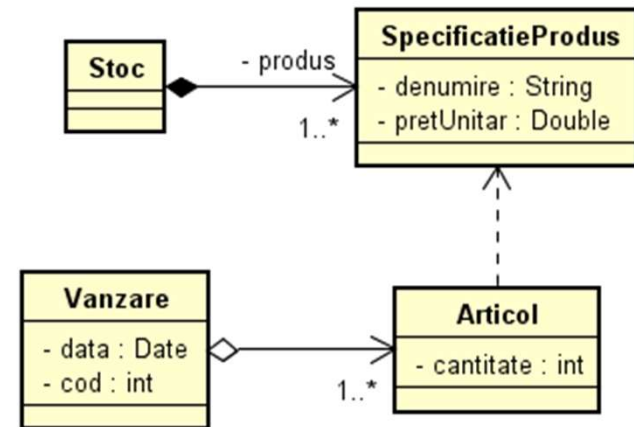


M.P. Information Expert (cont.)

Ce informatii sunt necesare pentru a calcula costul total al unei vanzari?

Trebuie sa stim care au fost produsele cumparate, pretul lor unitar si cantitatea cumparata.

$\text{costArticol} = \text{pretUnitarProdus} * \text{cantitate}$



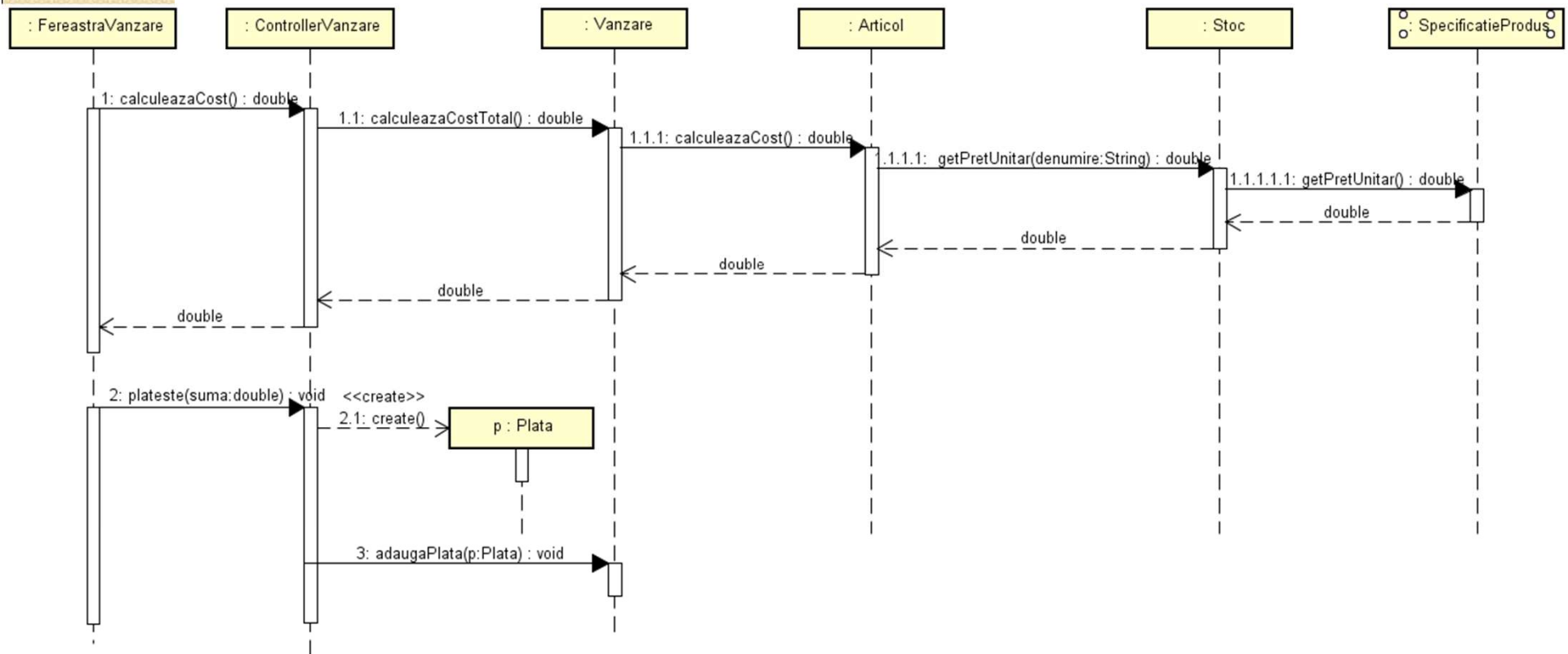
Clasa	Responsabilitati
SpecificatieProdus	-stie denumire, pretUnitar
	- da acces la pretul unitar al produsului curent (a face)
Articol	- stie cantitate
	- Calculeaza costul articolului current (a face)
	- Da acces la acest cost (a face)
Stoc	- Creeaza obiecte SpecificatieProdus (a face – m.p. Creator)
	- Furnizeaza pretul unitar al fiecarui produs cerute de Articol (a face)
Vanzare	- Creaza obiecte Articol (a face = mp. Creator)
	- Calculeaza costul total al vanzarii (a face)



M.P. Low Coupling

Problema: Cum realizam proiectarea astfel incat sa avem cat mai putine legaturi de dependenta intre clase, impactul modificarii sa fie mic si gradul de reutilizare sa fie crescut?

Solutie: Alocam o responsabilitate unei clase astfel incat cuplarea sa raman joasa.
Varianta 1: urmatoarea diagrama de secventa:



Cuplare(FereastraVanzare) = 1

Cuplare(ControllerVanzare) = 2

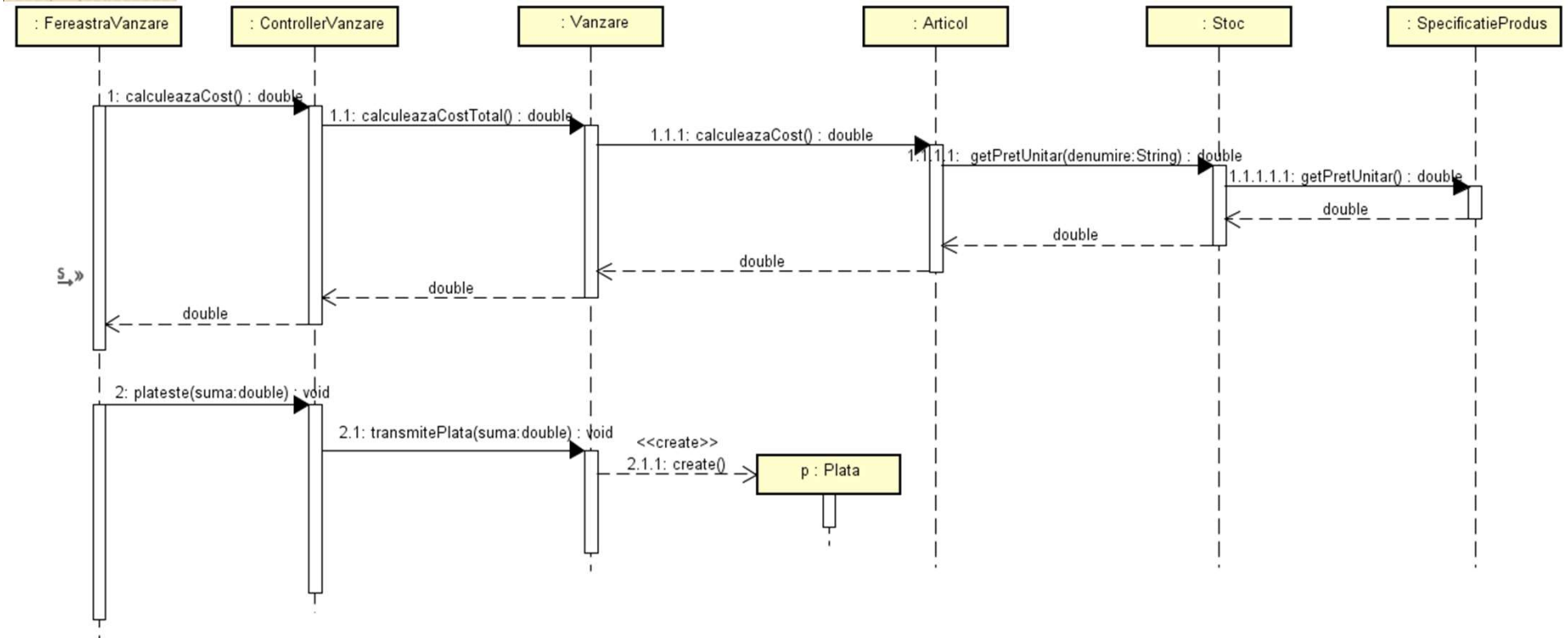
Cuplare(Vanzare)=2

Cuplare(Articol)=1

Cuplare(Stoc)=1

Cuplare(SpecificatieProdus)=1

M.P. Low Coupling



Cuplare(FereastraVanzare) = 1
Cuplare(ControllerVanzare) = 1
Cuplare(Vanzare)=2
Cuplare(Articol)=1
Cuplare(Stoc)=1

Concluzie: Alegem varianta a doua.

M.P. Pure Fabrication (Inventie)

Problema: Ce clasă ar trebui să aibă o responsabilitate când nu vrem să încălcăm principiile de proiectare de Coeziune înaltă și Cuplare slabă?

Există situații când atribuirea responsabilităților doar claselor care provin din domeniul problemei ar duce la probleme de cuplare înaltă sau de coeziune joasă sau grad de reutilizare mic.

Soluție. Responsabilitatea este atribuită unei clase software artificială pentru a îndeplini principiile de coeziune înaltă, cuplare slabă și reutilizare crescută.

Exemplu. Cine este responsabil cu salvarea datelor din obiectele clasei Vanzare? Conform m.p. Expert este clasa Vanzare, dar în aceste caz, am încălca principiile de coeziune înaltă și cuplare slabă.

Soluție. Se introduce clasa ManagerVanzare care se ocupă cu gestiunea datelor despre vânzare pe suport persistent (bd sau/si fisiere).

ManagerVanzare
+ memoreaza(v : Vanzare) : boolean + getVanzare(cod : int) : Vanzare + stergeVanzare(cod : int) : boolean



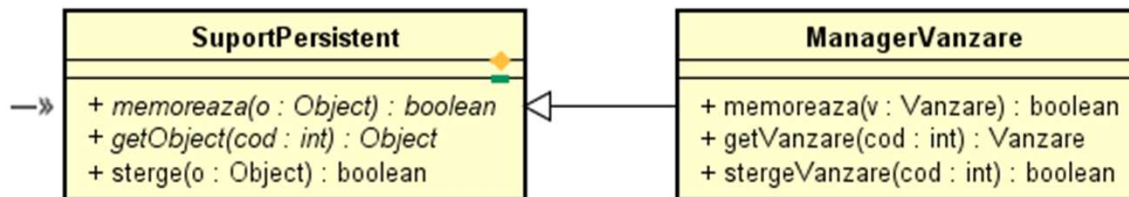
M.P. Indirectare

Problema: Cărei clase atribuim o responsabilitate pentru a evita cuplarea directă între două sau mai multe clase?

Soluție: Atribuim responsabilitatea unei clase intermediar care face legătura între clase.

Exemplu: Intr-o arhitectură software există mai multe clase ale căror date trebuie să fie memorate pe suport persistent. În acest caz, dacă toate clasele au aceleași responsabilități, putem face o singură clasă abstractă SuportPersistent.

Clasa SuportPersistent acționează ca un intermediar ce face legătura între clasele Manager și celelalte clase.



M.P. Polimorfism

Probleme:

1. Cum putem crea componente software sau clase reutilizabile? Adica vazand clasele si obiectele in roluri de client/furnizor, se pune problema cum putem inlocui o clasa sau obiect furnizor fara ca obiectul client sa fie afectat?
2. Cum proiectam atunci cand avem comportamente asemanatoare, dar diferite intre ele in functie de tipul (clasa) in care se gaseste? Sau cum proiectam OO o responsabilitate rezolvata in mod diferit in clase diferite?

Solutia in proiectarea OO este de a declara comportamentul intr-o interfata ce va fi realizata de clasele care vor realiza (implementa) comportamentul declarat.

